

A/B Test Analysis – E-Commerce Landing Page

This notebook covers a full A/B test project including:

- Data cleaning
- Conversion rate comparison
- Sampling distribution
- Z-test
- Logistic regression
- Confidence intervals
- Power analysis
- Time-based conversion trends

```
In [7]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
import statsmodels.stats.api as sms
import datetime as dt

# Load data
ab_test = pd.read_csv('ab_test.csv')
countries = pd.read_csv('countries_ab.csv')

# Preprocessing
ab_test.columns = ['user_id', 'timestamp', 'group', 'landing_page', 'converted']
countries.columns = ['user_id', 'country']
ab_test['user_id'] = ab_test['user_id'].astype(str)
countries['user_id'] = countries['user_id'].astype(str)
df = ab_test.merge(countries, on='user_id', how='left')

# Filter and clean data
df = df[((df['group'] == 'treatment') & (df['landing_page'] == 'new_page')) |
        ((df['group'] == 'control') & (df['landing_page'] == 'old_page'))]
df = df.drop_duplicates(subset='user_id')
df['intercept'] = 1
df['ab_page'] = df['group'].apply(lambda x: 1 if x == 'treatment' else 0)
df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')
```

```
In [8]: # Conversion stats
control_mean = df[df['group'] == 'control']['converted'].mean()
treat_mean = df[df['group'] == 'treatment']['converted'].mean()
obs_diff = treat_mean - control_mean

# Confidence interval
n_control = df[df['group'] == 'control'].shape[0]
n_treatment = df[df['group'] == 'treatment'].shape[0]
p_pool = (control_mean * n_control + treat_mean * n_treatment) / (n_control + n_treatment)
se_diff = np.sqrt(p_pool * (1 - p_pool) * (1/n_control + 1/n_treatment))
conf_int = (obs_diff - 1.96 * se_diff, obs_diff + 1.96 * se_diff)
conf_int
```

Out[8]: (-0.00393790891517839, 0.0007814309444672761)

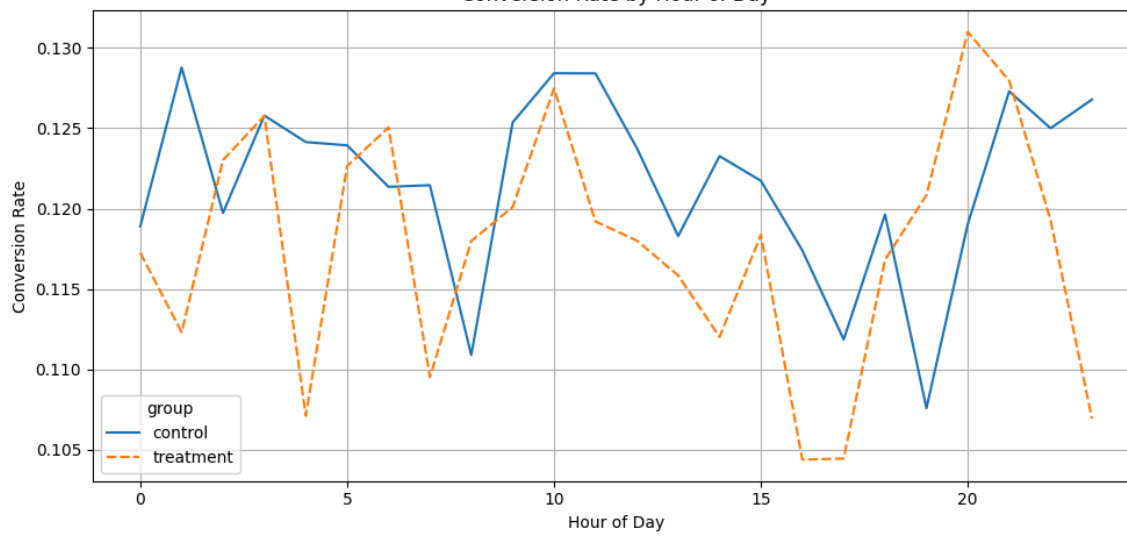
```
In [9]: # Power analysis
effect_size = sms.proportion_effectsize(treat_mean, control_mean)
power_analysis = sms.NormalIndPower().power(
    effect_size=effect_size,
    nobs1=n_control,
    alpha=0.05,
    ratio=n_treatment / n_control,
    alternative='two-sided'
)
power_analysis
```

Out[9]: 0.25869435956523174

```
In [10]: # Time-based conversion analysis
df['hour'] = df['timestamp'].dt.hour
df['date'] = df['timestamp'].dt.date
conversion_by_hour = df.groupby(['hour', 'group'])['converted'].mean().unstack()
conversion_by_date = df.groupby(['date', 'group'])['converted'].mean().unstack()

# Plot hourly trends
plt.figure(figsize=(10, 5))
sns.lineplot(data=conversion_by_hour)
plt.title('Conversion Rate by Hour of Day')
plt.ylabel('Conversion Rate')
plt.xlabel('Hour of Day')
plt.grid(True)
plt.tight_layout()
plt.show()
```

Conversion Rate by Hour of Day



Recommendations for Future A/B Testing

1. Increase Sample Size

- Test power was only ~26%, below the standard 80%.
- Use a sample size calculator and power analysis before launching.

2. Pre-Experiment Design Checklist

- Define Minimum Detectable Effect (MDE).
- Estimate required sample size and duration.
- Ensure randomization and group balancing.

3. Segment-Based Analysis

- Analyze results by user type, device, traffic source.
- You might find significant effects in user subgroups.

4. Improve Experiment Sensitivity

- Include continuous metrics (e.g., revenue, time on page).
- Use covariates or CUPED for increased precision.

5. Test More Impactful Variants

- Larger design changes can yield bigger effects.
- Try variations in content, layout, or personalization.

6. Consider Sequential or Bayesian Testing

- Use techniques that allow early stopping without inflating error.
- Bayesian methods offer continuous insights into variant performance.

7. Report Effect Sizes Alongside P-values

- Small effects can be meaningful at scale.
- Helps stakeholders make informed business decisions.